Knowledge Graphs for System Integration Risk - Gunjan Rawat

Introduction -

Modern software systems are increasingly composed of large, interconnected networks of microservices, third-party APIs, storage layers, compute services, and communication channels. As these systems grow in scale and complexity, understanding integration risk, the likelihood that the failure of a single component will disrupt overall system performance becomes more essential. Traditional approaches to system risk analysis rely heavily on manual inspection or rule-based heuristics, all of which fail to scale or generalize to rapidly evolving architectures such as distributed cloud platforms, event-driven services, or machine-learning–backed pipelines.

This project explores a hybrid methodology that uses Large Language Models (LLMs) and Graph Neural Networks (GNNs) to automate integration-risk assessment across complex systems. The goal is to design a pipeline that can extract system components and their dependencies, construct a knowledge graph, compute heuristic risk features, train a GCN using these features, evaluate them on a big system and finally classify each component into low-, medium-, or high-risk categories. By combining LLMs with the reasoning strengths of GNNs, the project demonstrates how emerging AI techniques can significantly improve system-level risk modeling.

Methodology -

The methodology for this project follows a structured multi-stage pipeline that integrates Large Language Models (LLMs), graph construction techniques, synthetic labeling, and Graph Neural Networks (GNNs) to assess integration risk within complex system architectures. The full workflow is illustrated in the project pipeline diagram (Figure 1) and described in detail below.

1. System Description Generation

The pipeline begins with a prompt, where I specified 12 small systems (10-20 nodes and edges) to analyze (e.g., payment gateway, recommendation engine, media pipeline). This prompt is passed to a System Document Generator (parser.py), implemented using an LLM, which produces 2 csv files - 1 for the nodes and 1 for the edges (Figure 2).

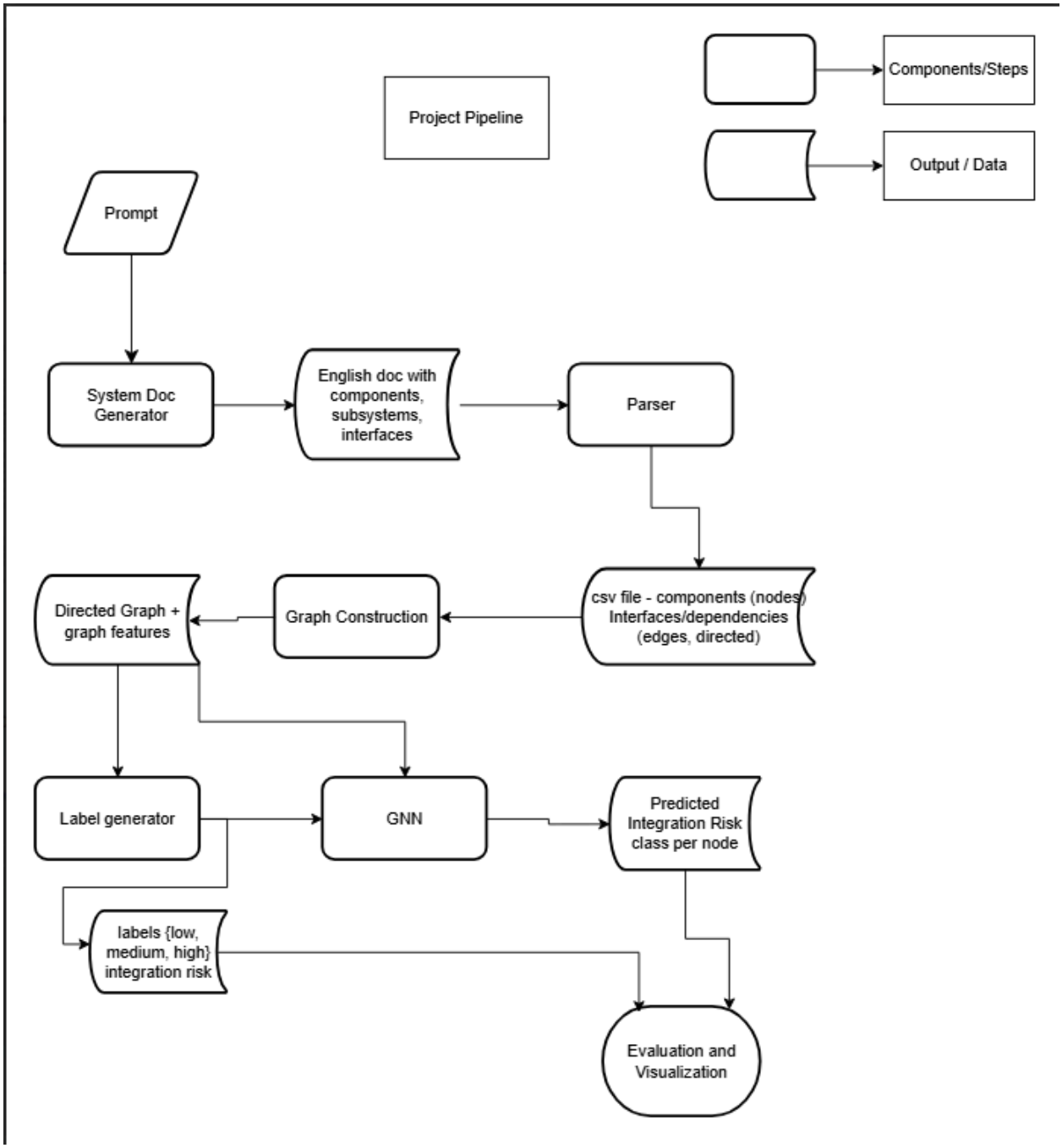


Figure 1 - Pipeline diagram

2. Parsing and Structured CSV Generation

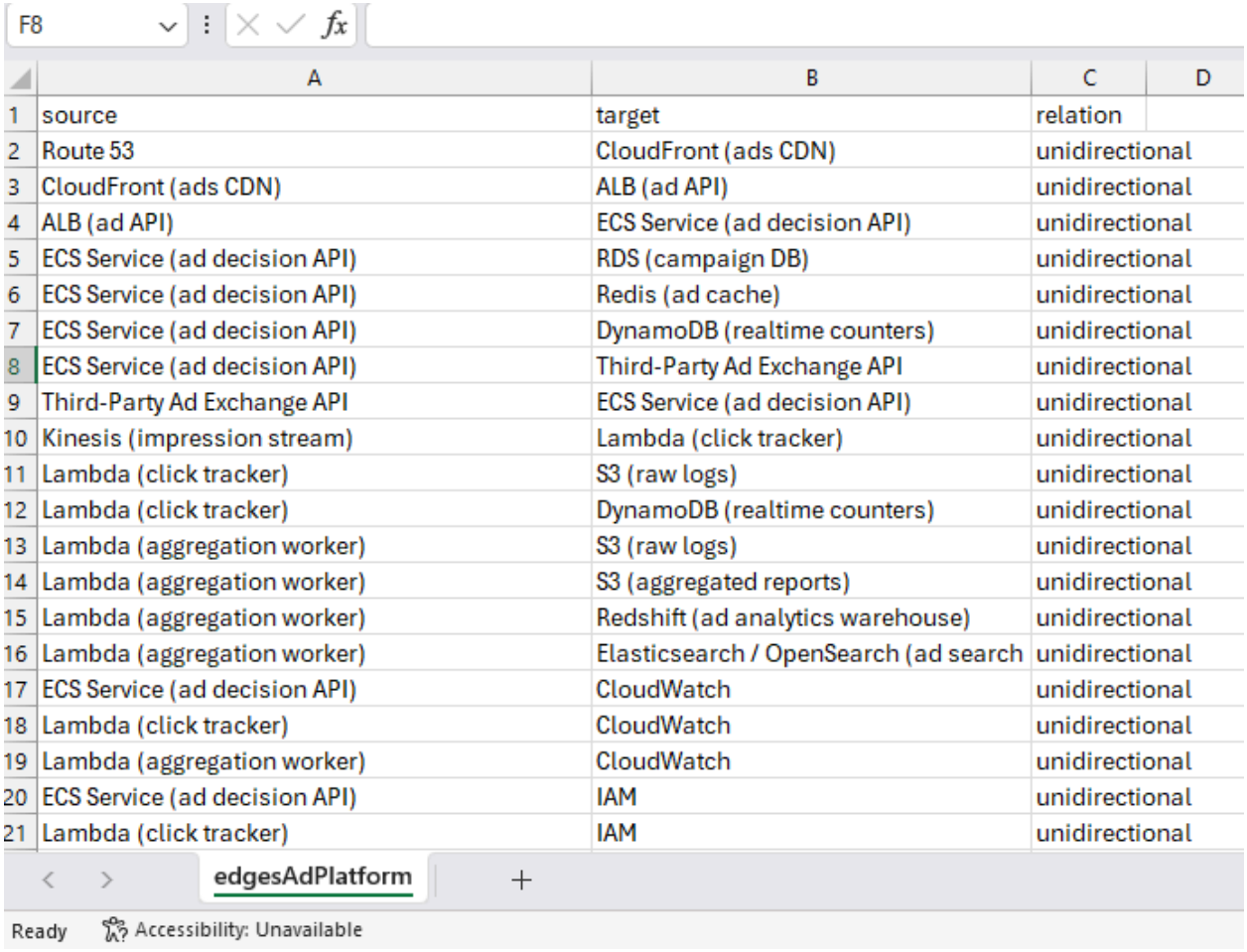
The generated English document is then fed into a Parser, which extracts structured system data.The parser outputs two CSV files:

- Components CSV - Each row represents a node (system component)
- Edges CSV - Each row represents a directed relation between components

	A	B	C	D	E	F
1	name					
2	Route 53					
3	CloudFront (ads CDN)					
4	ALB (ad API)					
5	ECS Service (ad decision API)					
6	RDS (campaign DB)					
7	DynamoDB (realtime counters)					
8	Kinesis (impression stream)					
9	Lambda (click tracker)					
10	Lambda (aggregation worker)					
11	S3 (raw logs)					
12	S3 (aggregated reports)					
13	Redshift (ad analytics warehouse)					
14	Elasticsearch / OpenSearch (ad search index)					
15	Redis (ad cache)					
16	Third-Party Ad Exchange API					
17	CloudWatch					
18	IAM					
19	VPC (main)					
20						
21						

components_adplatform

Fig 2.1 - Generated components.csv for the ad platform system



	A	B	C	D
1	source	target	relation	
2	Route 53	CloudFront (ads CDN)	unidirectional	
3	CloudFront (ads CDN)	ALB (ad API)	unidirectional	
4	ALB (ad API)	ECS Service (ad decision API)	unidirectional	
5	ECS Service (ad decision API)	RDS (campaign DB)	unidirectional	
6	ECS Service (ad decision API)	Redis (ad cache)	unidirectional	
7	ECS Service (ad decision API)	DynamoDB (realtime counters)	unidirectional	
8	ECS Service (ad decision API)	Third-Party Ad Exchange API	unidirectional	
9	Third-Party Ad Exchange API	ECS Service (ad decision API)	unidirectional	
10	Kinesis (impression stream)	Lambda (click tracker)	unidirectional	
11	Lambda (click tracker)	S3 (raw logs)	unidirectional	
12	Lambda (click tracker)	DynamoDB (realtime counters)	unidirectional	
13	Lambda (aggregation worker)	S3 (raw logs)	unidirectional	
14	Lambda (aggregation worker)	S3 (aggregated reports)	unidirectional	
15	Lambda (aggregation worker)	Redshift (ad analytics warehouse)	unidirectional	
16	Lambda (aggregation worker)	Elasticsearch / OpenSearch (ad search	unidirectional	
17	ECS Service (ad decision API)	CloudWatch	unidirectional	
18	Lambda (click tracker)	CloudWatch	unidirectional	
19	Lambda (aggregation worker)	CloudWatch	unidirectional	
20	ECS Service (ad decision API)	IAM	unidirectional	
21	Lambda (click tracker)	IAM	unidirectional	

Fig 2.2 - Generated edges.csv for the ad platform system

3. Graph Construction and Feature Extraction

For each system configuration in the SYSTEMS list, the pipeline constructs a directed graph using NetworkX and then extracts structural features for every component.

First, the components CSV and edges CSV for a given system_id are loaded into pandas dataframe. A directed graph G (NetworkX DiGraph) is created where:

- Each row in the components file contributes a node whose identifier is the component's name.
- Each row in the edges file contributes a directed edge from source to target.

Once the graph is built, the pipeline computes a set of node-level features:

- Degree-based metrics
    - In-degree (in_degree)
    - Out-degree (out_degree)

    - Total degree (degree)

- Centrality measures
    - Betweenness centrality (betweenness)
    - Closeness centrality (closeness)
    - Eigenvector centrality (eigenvector) using eigenvector_centrality_numpy (with a safe fallback of 0.0 for all nodes if the computation fails)
    - PageRank (pagerank)

- Reachability metrics
    - downstream_count: number of descendants of each node (size of the set of nodes reachable via outgoing paths)
    - upstream_count: number of ancestors of each node (size of the set of nodes that can reach the node)

- Articulation property
    - The directed graph is projected to an undirected version and NetworkX's articulation_points is used to flag whether a node is an articulation point (is_articulation), (its removal would disconnect parts of the graph.)

These features are assembled into a pandas DataFrame with one row per node, augmented with the corresponding system_id. All per-system DataFrames are later concatenated into a single dataset and written to **all_systems_node_features.csv**, which serves as the main input to the GNN model. In parallel, all edge files are merged into **all_systems_edges.csv** for reproducibility and visualization.

4. Heuristic Risk Scoring

Synthetic integration-risk labels are generated directly from the computed node features using a **quantile-based importance score**. The score is independently calculated for each system.

First, three core indicators are selected and **min–max normalized** within the system:

- Total degree (degree)
- Betweenness centrality (betweenness)
- Downstream reachability (downstream_count)

Normalization uses a simple linear scale to [0, 1]; in the edge case where all values are identical, the normalized score is set to 0.0 for all nodes.

An **articulation bonus** is then derived from the Boolean is_articulation flag:

- art_bonus = 1.0 if the node is an articulation point
- art_bonus = 0.0 otherwise

The overall **importance score** for each node is computed as a weighted sum:

$$\textbf{importance\_score} = 0.4 \cdot \textbf{deg\_norm} + 0.4 \cdot \textbf{bet\_norm} + 0.4 \cdot \textbf{down\_norm} + 0.2 \cdot \textbf{art\_bonus}$$

This score captures both local connectivity (degree), global positioning (betweenness), potential blast radius (downstream_count), and structural criticality (articulation).

To convert this continuous score into risk classes, the pipeline computes two quantiles of the **importance_score** distribution for each system:

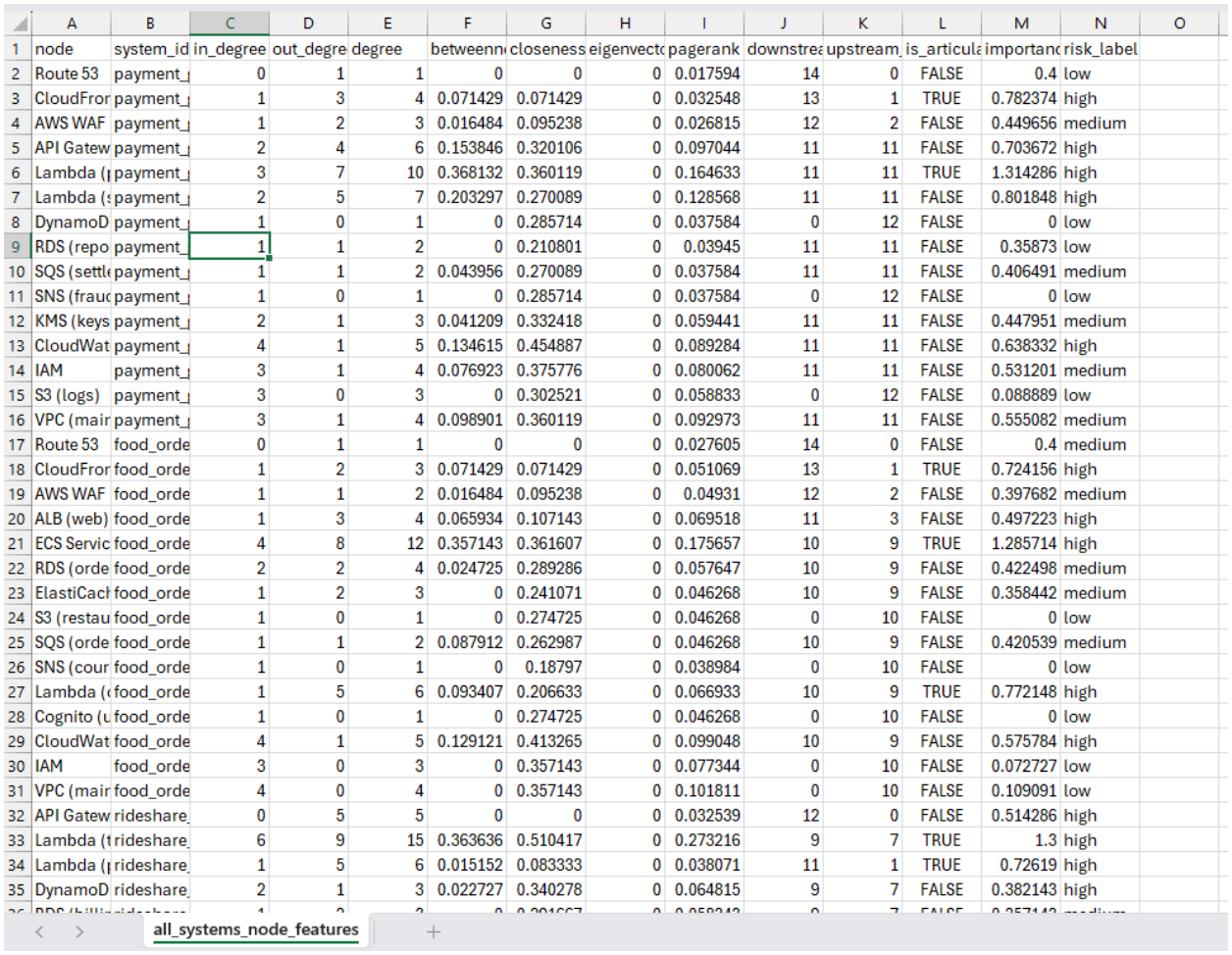
- q_low = 0.33 quantile
- q_high = 0.66 quantile

Each node is then assigned a **synthetic risk label**:

- **“high”** risk if importance_score ≥ q_high
- **“low”** risk if importance_score ≤ q_low
- **“medium”** risk otherwise

The resulting risk_label column is stored alongside the node features and used for training the GNN. This design yields labels that are:

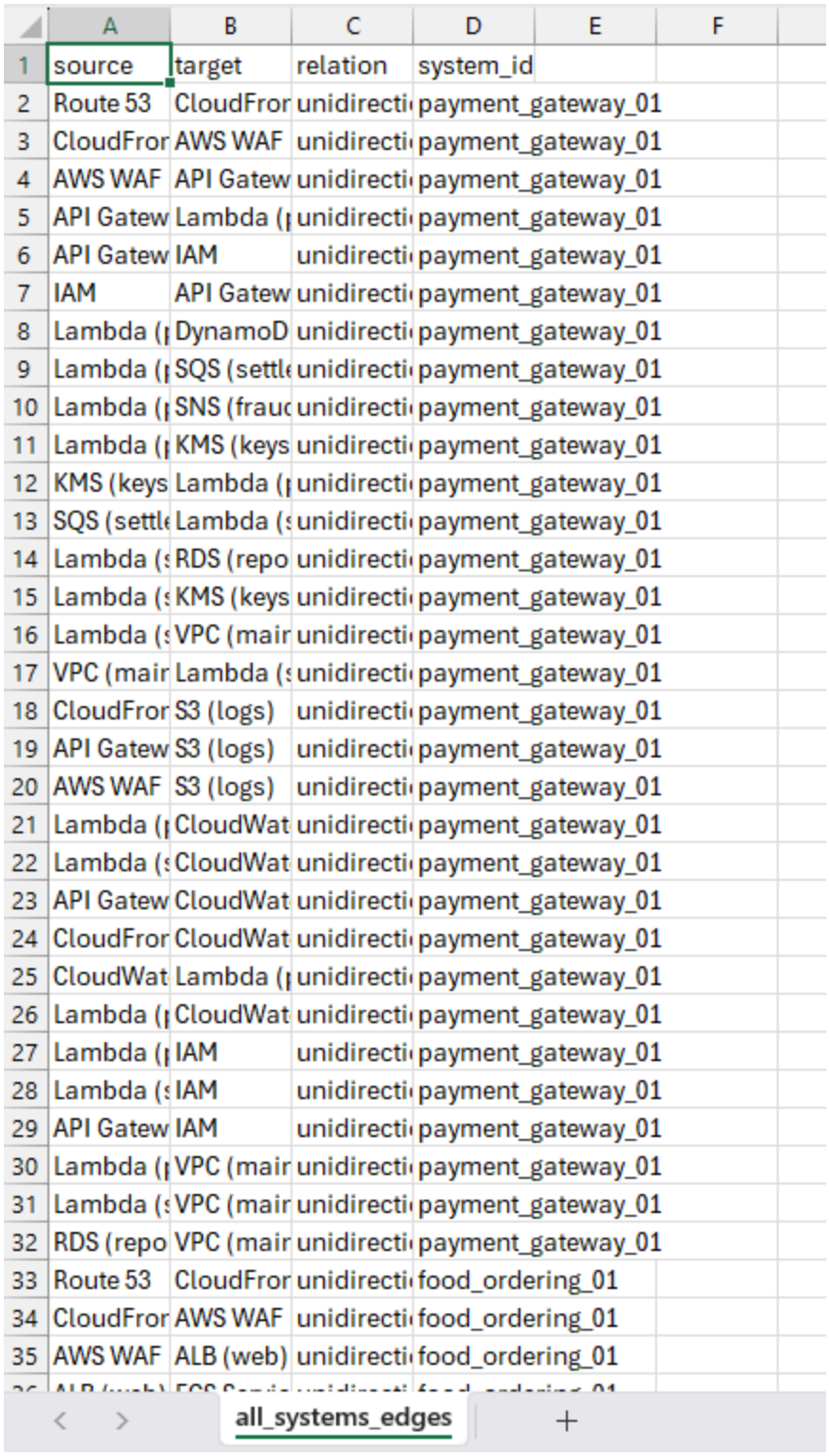
- **System-relative** (adapted to each architecture’s scale and shape)
- **Simple to interpret**, making them suitable both for training and for heuristic baselines when comparing against the learned GNN predictions.



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	node	system_id	in_degree	out_degre	degree	betweenn	closeness	eigenvecto	pagerank	downstrea	upstream_	is_articula	importanc	risk_label	
2	Route 53	payment_	0	1	1	0	0	0	0.017594	14	0	FALSE	0.4	low	
3	CloudFror	payment_	1	3	4	0.071429	0.071429	0	0.032548	13	1	TRUE	0.782374	high	
4	AWS WAF	payment_	1	2	3	0.016484	0.095238	0	0.026815	12	2	FALSE	0.449656	medium	
5	API Gatew	payment_	2	4	6	0.153846	0.320106	0	0.097044	11	11	FALSE	0.703672	high	
6	Lambda (p	payment_	3	7	10	0.368132	0.360119	0	0.164633	11	11	TRUE	1.314286	high	
7	Lambda (s	payment_	2	5	7	0.203297	0.270089	0	0.128568	11	11	FALSE	0.801848	high	
8	DynamoD	payment_	1	0	1	0	0.285714	0	0.037584	0	12	FALSE	0	low	
9	RDS (repo	payment_	1	1	2	0	0.210801	0	0.03945	11	11	FALSE	0.35873	low	
10	SQS (settl	payment_	1	1	2	0.043956	0.270089	0	0.037584	11	11	FALSE	0.406491	medium	
11	SNS (fraud	payment_	1	0	1	0	0.285714	0	0.037584	0	12	FALSE	0	low	
12	KMS (keys	payment_	2	1	3	0.041209	0.332418	0	0.059441	11	11	FALSE	0.447951	medium	
13	CloudWat	payment_	4	1	5	0.134615	0.454887	0	0.089284	11	11	FALSE	0.638332	high	
14	IAM	payment_	3	1	4	0.076923	0.375776	0	0.080062	11	11	FALSE	0.531201	medium	
15	S3 (logs)	payment_	3	0	3	0	0.302521	0	0.058833	0	12	FALSE	0.088889	low	
16	VPC (mair	payment_	3	1	4	0.098901	0.360119	0	0.092973	11	11	FALSE	0.555082	medium	
17	Route 53	food_orde	0	1	1	0	0	0	0.027605	14	0	FALSE	0.4	medium	
18	CloudFror	food_orde	1	2	3	0.071429	0.071429	0	0.051069	13	1	TRUE	0.724156	high	
19	AWS WAF	food_orde	1	1	2	0.016484	0.095238	0	0.04931	12	2	FALSE	0.397682	medium	
20	ALB (web)	food_orde	1	3	4	0.065934	0.107143	0	0.069518	11	3	FALSE	0.497223	high	
21	ECS Servic	food_orde	4	8	12	0.357143	0.361607	0	0.175657	10	9	TRUE	1.285714	high	
22	RDS (orde	food_orde	2	2	4	0.024725	0.289286	0	0.057647	10	9	FALSE	0.422498	medium	
23	ElastiCach	food_orde	1	2	3	0	0.241071	0	0.046268	10	9	FALSE	0.358442	medium	
24	S3 (restau	food_orde	1	0	1	0	0.274725	0	0.046268	0	10	FALSE	0	low	
25	SQS (orde	food_orde	1	1	2	0.087912	0.262987	0	0.046268	10	9	FALSE	0.420539	medium	
26	SNS (cour	food_orde	1	0	1	0	0.18797	0	0.038984	0	10	FALSE	0	low	
27	Lambda (c	food_orde	1	5	6	0.093407	0.206633	0	0.066933	10	9	TRUE	0.772148	high	
28	Cognito (u	food_orde	1	0	1	0	0.274725	0	0.046268	0	10	FALSE	0	low	
29	CloudWat	food_orde	4	1	5	0.129121	0.413265	0	0.099048	10	9	FALSE	0.575784	high	
30	IAM	food_orde	3	0	3	0	0.357143	0	0.077344	0	10	FALSE	0.072727	low	
31	VPC (mair	food_orde	4	0	4	0	0.357143	0	0.101811	0	10	FALSE	0.109091	low	
32	API Gatew	rideshare_	0	5	5	0	0	0	0.032539	12	0	FALSE	0.514286	high	
33	Lambda (t	rideshare_	6	9	15	0.363636	0.510417	0	0.273216	9	7	TRUE	1.3	high	
34	Lambda (p	rideshare_	1	5	6	0.015152	0.083333	0	0.038071	11	1	TRUE	0.72619	high	
35	DynamoD	rideshare_	2	1	3	0.022727	0.340278	0	0.064815	9	7	FALSE	0.382143	high	

all_systems_node_features

Fig 3.1 - all_system_node_features.csv



	A	B	C	D	E	F
1	source	target	relation	system_id		
2	Route 53	CloudFror	unidirecti	payment_gateway_01		
3	CloudFror	AWS WAF	unidirecti	payment_gateway_01		
4	AWS WAF	API Gatew	unidirecti	payment_gateway_01		
5	API Gatew	Lambda (p	unidirecti	payment_gateway_01		
6	API Gatew	IAM	unidirecti	payment_gateway_01		
7	IAM	API Gatew	unidirecti	payment_gateway_01		
8	Lambda (p	DynamoD	unidirecti	payment_gateway_01		
9	Lambda (p	SQS (settle	unidirecti	payment_gateway_01		
10	Lambda (p	SNS (frauc	unidirecti	payment_gateway_01		
11	Lambda (p	KMS (keys	unidirecti	payment_gateway_01		
12	KMS (keys	Lambda (p	unidirecti	payment_gateway_01		
13	SQS (settle	Lambda (s	unidirecti	payment_gateway_01		
14	Lambda (s	RDS (repo	unidirecti	payment_gateway_01		
15	Lambda (s	KMS (keys	unidirecti	payment_gateway_01		
16	Lambda (s	VPC (mair	unidirecti	payment_gateway_01		
17	VPC (mair	Lambda (s	unidirecti	payment_gateway_01		
18	CloudFror	S3 (logs)	unidirecti	payment_gateway_01		
19	API Gatew	S3 (logs)	unidirecti	payment_gateway_01		
20	AWS WAF	S3 (logs)	unidirecti	payment_gateway_01		
21	Lambda (p	CloudWat	unidirecti	payment_gateway_01		
22	Lambda (s	CloudWat	unidirecti	payment_gateway_01		
23	API Gatew	CloudWat	unidirecti	payment_gateway_01		
24	CloudFror	CloudWat	unidirecti	payment_gateway_01		
25	CloudWat	Lambda (p	unidirecti	payment_gateway_01		
26	Lambda (p	CloudWat	unidirecti	payment_gateway_01		
27	Lambda (p	IAM	unidirecti	payment_gateway_01		
28	Lambda (s	IAM	unidirecti	payment_gateway_01		
29	API Gatew	IAM	unidirecti	payment_gateway_01		
30	Lambda (p	VPC (mair	unidirecti	payment_gateway_01		
31	Lambda (s	VPC (mair	unidirecti	payment_gateway_01		
32	RDS (repo	VPC (mair	unidirecti	payment_gateway_01		
33	Route 53	CloudFror	unidirecti	food_ordering_01		
34	CloudFror	AWS WAF	unidirecti	food_ordering_01		
35	AWS WAF	ALB (web)	unidirecti	food_ordering_01		

Fig 3.2 - all_system_edges.csv

5. Graph Conversion to pytorch format

After computing features and heuristic risk labels for each node, the combined node and edge tables (all_systems_node_features.csv and all_systems_edges.csv) are converted into a collection of PyTorch Geometric graphs. This is implemented in make_pyg_graphs.py via the load_system_graphs function.

For node features, these columns are selected:
in_degree, out_degree, degree,betweenness, closeness, eigenvector, pagerank,downstream_count, upstream_count,is_articulation, importance_score

6. GNN Architecture, Training, and Evaluation

The node classification model is a **two-layer Graph Convolutional Network (GCN)** implemented in trainGNN.py as the RiskGCN class. The architecture is:

1. GCNConv(in_dim, hidden_dim)
2. ReLU nonlinearity
3. GCNConv(hidden_dim, hidden_dim)
4. ReLU nonlinearity
5. Final linear layer Linear(hidden_dim, num_classes)

in_dim is the dimensionality of the node feature vector (11 features),hidden_dim is set to 32,num_classes is 3 (low, medium, high risk).

For training, all loaded system graphs are split at the **system level**:

- The sorted list of system_ids is split into:
    - First 8 systems → **training set**
    - Remaining 4 systems → **validation set**

Two routines, train_epoch and eval_epoch, compute:

- Average loss per node
- Overall node-level accuracy

After training, the model parameters are saved

```
Epoch 020 | train_loss 0.849 | train_acc 0.617 | val_loss 1.150 | val_acc 0.475
Epoch 040 | train_loss 0.777 | train_acc 0.635 | val_loss 1.107 | val_acc 0.424
Epoch 060 | train_loss 0.731 | train_acc 0.652 | val_loss 1.092 | val_acc 0.441
Epoch 080 | train_loss 0.690 | train_acc 0.661 | val_loss 1.096 | val_acc 0.424
Epoch 100 | train_loss 0.646 | train_acc 0.687 | val_loss 1.115 | val_acc 0.458
Epoch 020 | train_loss 0.849 | train_acc 0.617 | val_loss 1.150 | val_acc 0.475
Epoch 040 | train_loss 0.777 | train_acc 0.635 | val_loss 1.107 | val_acc 0.424
Epoch 060 | train_loss 0.731 | train_acc 0.652 | val_loss 1.092 | val_acc 0.441
Epoch 080 | train_loss 0.690 | train_acc 0.661 | val_loss 1.096 | val_acc 0.424
Epoch 100 | train_loss 0.646 | train_acc 0.687 | val_loss 1.115 | val_acc 0.458
Epoch 120 | train_loss 0.603 | train_acc 0.696 | val_loss 1.131 | val_acc 0.458
Epoch 140 | train_loss 0.558 | train_acc 0.713 | val_loss 1.120 | val_acc 0.441
Epoch 160 | train_loss 0.514 | train_acc 0.809 | val_loss 1.148 | val_acc 0.525
Epoch 180 | train_loss 0.475 | train_acc 0.826 | val_loss 1.214 | val_acc 0.492
Epoch 200 | train_loss 0.429 | train_acc 0.826 | val_loss 1.280 | val_acc 0.525
Saved model to risk_gcn_system_split.pth
```

Fig 4 - Training the GCN

Finally, I used the trained model to predict labels on the Main system (60 + components and nodes)

7. Results

Training and Validation Performance :

The GNN was trained for 200 epochs on eight systems and validated on four unseen systems using node-level cross-entropy loss and accuracy as metrics. Figure 4 (training log) summarizes the evolution of both metrics across training.

The model demonstrates steady improvement throughout training:

- Training loss decreases consistently from ~0.85 to ~0.43
- Training accuracy improves from ~0.62 to ~0.83

This pattern indicates that the GNN is successfully learning useful structural patterns from the node features and dependency graph. Because synthetic labels are derived from degree, betweenness, reachability, and articulation metrics, the GNN effectively learns to approximate these heuristics.

Validation Performance :

Validation accuracy is in the range 0.42–0.52, with final accuracy approximately 0.52. Unlike the training set, validation graphs come from entirely different system architectures, meaning the GNN must generalize its learned structural heuristics to unfamiliar dependency topologies. The model maintains stable performance, suggesting:

- Generalization is occurring, though it is not very perfect

- Performance is reasonable given the small dataset (12 graphs total)
- Validation accuracy above random-guess baseline (1/3 = 0.33)
- Variability across epochs is expected due to small validation batch size (only 4 graphs)

The gap between training and validation accuracy reflects a mild overfitting trend. The model achieves an improvement of nearly 20 percentage points over baseline, confirming that structural node features combined with GNN message passing yield a usable predictive signal.

Heuristic risk patterns for the small systems -

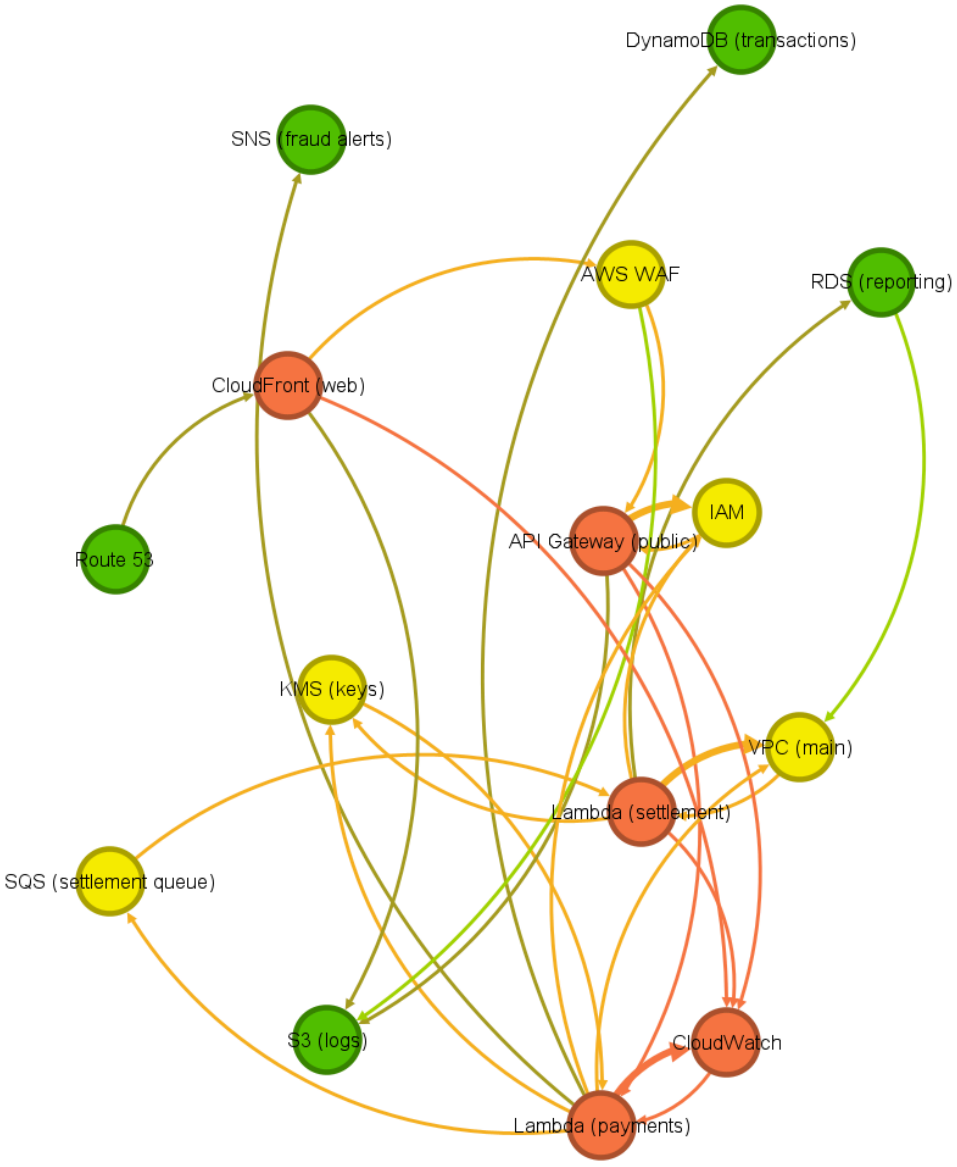


Fig 5 - Heuristic labels on the payment_gateway system

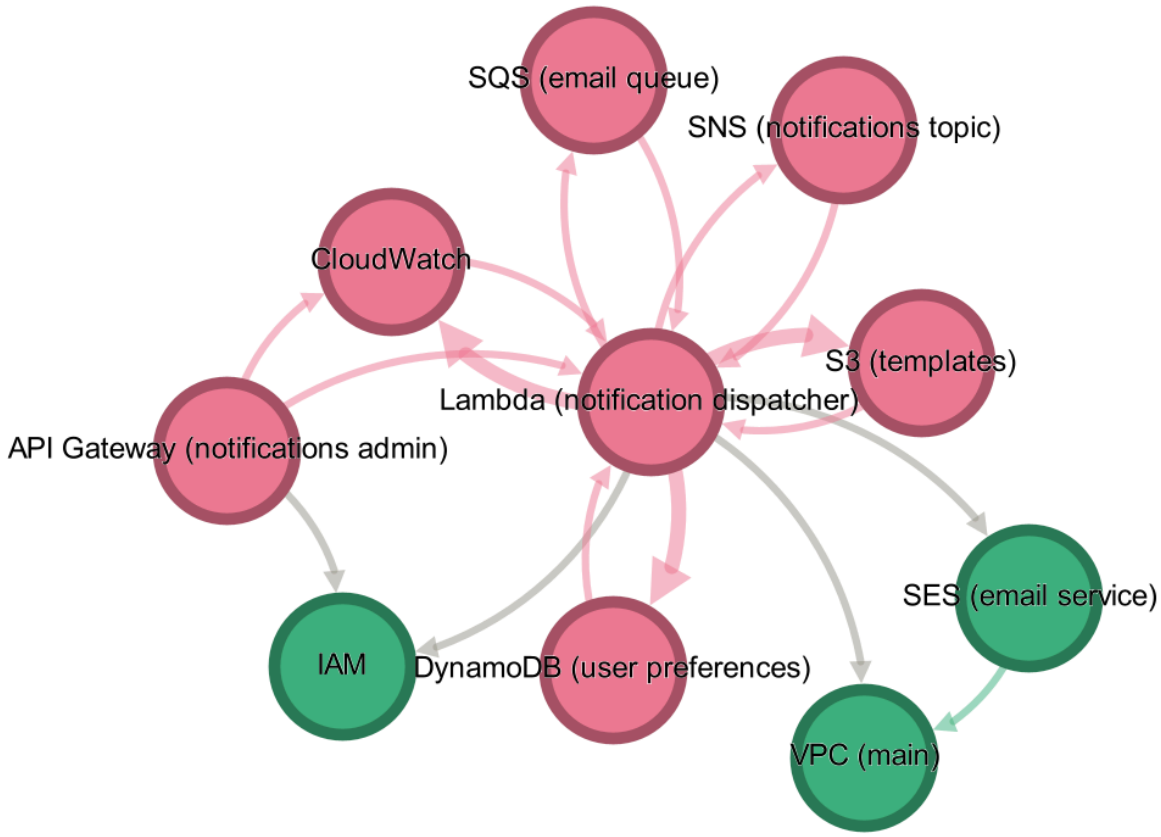


Fig 6 - Heuristic labels on the notification_service system

Across the smaller systems (notification service and payment gateway), the heuristic method produces risk scores that generally make sense based on the structure of each graph.

Central components are labeled high-risk - Nodes that sit in the middle of the system, especially Lambda functions that route or process requests are marked as high-risk because many other components depend on them. Example: **Lambda (notification dispatcher)** and **Lambda (payments)**.

Messaging and routing layers show medium–high risk- Services like SQS, SNS, and CloudFront have many connections, so the heuristics treat them as important parts of the workflow. They often appear in the medium or high category.

Backend storage and infrastructure are shown as lower-risk- Components such as DynamoDB, RDS, IAM, and VPC usually have fewer connections and do not act as bottlenecks.These are marked low-risk because they are structurally peripheral.

The pattern is imperfect at places- The heuristics correctly identify where work is coordinated, but they sometimes overestimate risk for AWS-managed services (like SQS or CloudWatch) simply because they have many edges.

Heuristic and Predicted Labels on the Main System :

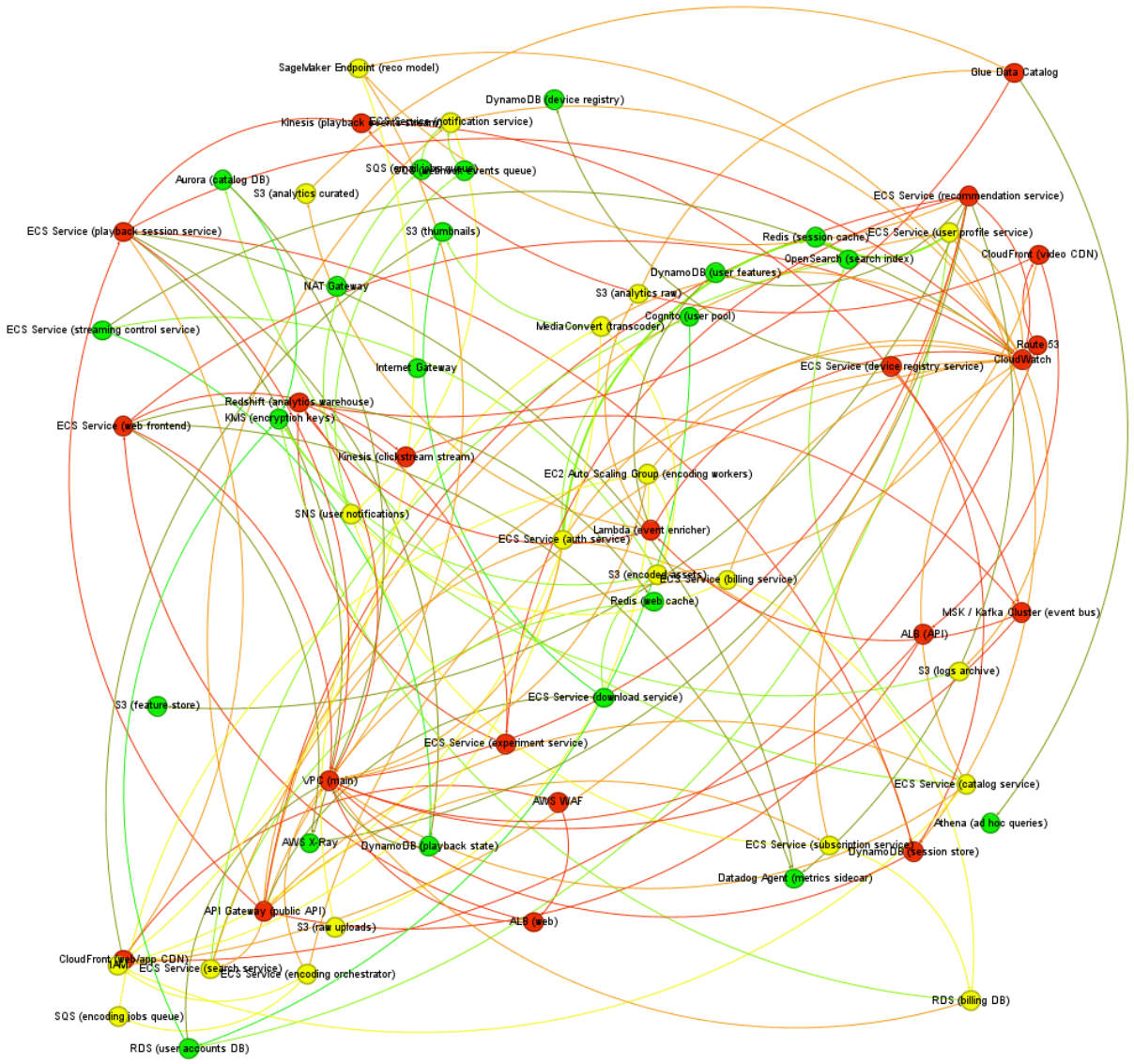


Fig 7 - Heuristic Labelling for Main System

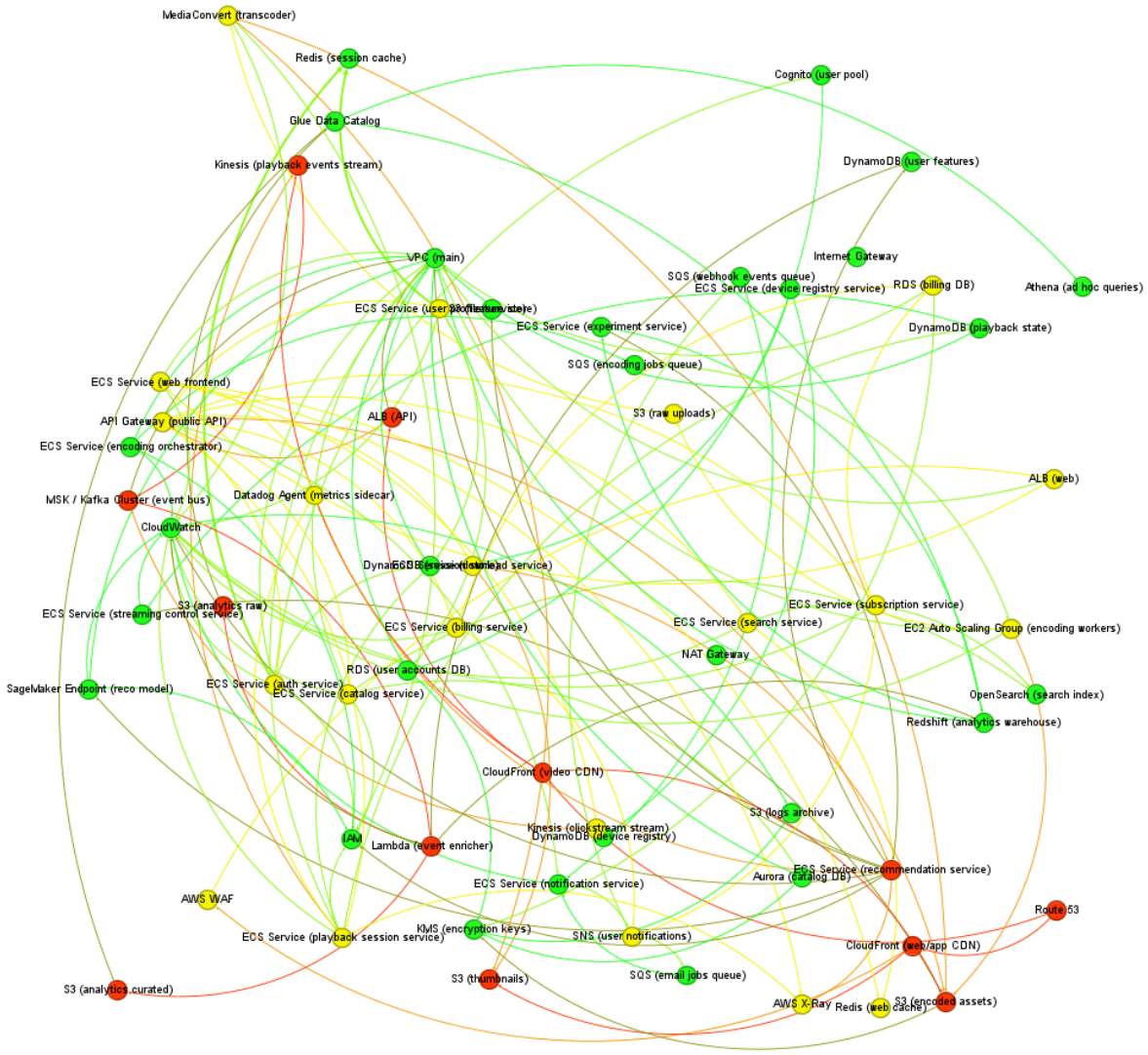


Fig 8 - GCN predicted label for the Main System

The heuristic graph marks many nodes as high-risk

In the heuristic version (Fig 7), A large portion of the system is colored red and many AWS components such as ECS services, S3 buckets, CloudFront, and Lambda functions appear as high-risk. The heuristic flags anything with many connections or a central position in the graph as **high risk**

The predicted graph is much more balanced in comparison.In the GNN prediction (Fig 8), Many components move from high-risk → medium or low risk. The graph looks more evenly distributed, with only a few true hotspots. Green nodes are more in number, meaning the GNN does **not** treat high connectivity alone as a failure risk.This shows that the GNN learned to **tone down false positives** from the heuristic.

The GNN focuses risk on actual coordination hubs

In the predicted graph, only a handful of core processing nodes (e.g., ECS services, certain Lambdas) remain in the high-risk group.Support services, monitoring services, data stores, and CDN layers shift to **medium or low risk**, because the GNN sees they are not structurally bottlenecks. **The GNN acts more selective**, picking out the real choke points rather than marking everything.

Infrastructure services become safer under the GNN

- CloudFront
- S3 buckets
- VPC
- IAM
- Metric/monitoring agents

All appear high-risk in the heuristic graph, but **low-risk in the predicted graph**.This reflects a more realistic model: these components are widely used but generally not fragile or central to workflow coordination.

Overall, the GNN produces a more realistic and interpretable risk map

- **Heuristic:** very noisy, many reds, overly sensitive to degree/betweenness
- **GNN:** smoother, cleaner, highlights only true dependency hotspots

8. What Improvements can be made -

Several enhancements could further improve accuracy and generalization

1. Expanding the dataset (more systems, more variation)- The GNN is currently trained on a relatively small number of system graphs. This limits its ability to generalize to architectures. More training diversity will significantly improve performance and stability.

2. Including edge features - Right now, the GNN only learns from nodes and the existence of edges. But from edges we can get :

- Data flow vs. control flow
- Internal traffic vs. third-party API calls

3. Making node features richer and more semantic- All current features are purely structural, But systems also contain **semantic categories**, such as:

- Databases
- Compute units
- Caches
- Messaging layers
- Public-facing endpoints

9. Conclusion

This project demonstrated a complete end-to-end pipeline for assessing integration risk in complex system architectures by combining Large Language Models, graph analytics, and Graph Neural Networks. Starting from natural-language system descriptions, the workflow automatically extracted components and dependencies, constructed directed graphs with structural features, generated heuristic risk labels, and trained a GNN to predict integration-risk levels for each component.

The results show that the GNN learns meaningful structural patterns beyond the raw heuristics. While the heuristic model tends to overestimate risk, especially for highly connected but operationally stable services, the GNN produces a more balanced and realistic risk distribution. It highlights coordination hubs and de-emphasizes peripheral infrastructure. The validation accuracy, although moderate, exceeds baseline, indicating real generalization across unseen system topologies.

Overall, the project validates that combining LLM-based extraction with graph learning provides a scalable and interpretable approach for system-level risk analysis. The framework is modular, extensible, and capable of supporting richer semantics, additional system types, and more advanced GNN architectures. With further improvements in dataset diversity and labeling quality, this approach can evolve into a tool for automated reliability assessment in modern distributed systems.

Prompts Used -

Model Used Chat GPT - 5.1

Generate a detailed system architecture description for a payment gateway, including components, subsystems, and all directed dependencies between them.

Rewrite the system description in a structured format listing components and the interfaces between them in simple English.

Create a more complex version of this architecture with 60 components and 100 edges while keeping dependencies realistic.

Given this system description, check if my parser is extracting all components and dependencies and output them as two lists: nodes and directed edges.

Format the system into a consistent structure so that my parser can detect components, edges, and bidirectional relationships reliably.

Is my parser converting the architecture into a CSV-ready format with columns: name, source, target, relation.

What graph metrics should I compute to estimate component-level risk in a dependency network?

Explain centrality metrics like degree, betweenness, closeness, and how they relate to system fragility.

How can I determine articulation points and descendant counts for nodes in a directed system graph?

Can you help me design a simple rule-based scoring function for assigning low, medium, and high integration-risk labels using graph features.

How can I normalize graph metrics so they can be combined into a unified importance score?

How do I convert multiple independent system graphs into PyTorch Geometric Data objects for node classification?

What GNN architecture should I use to classify risk at the node level, given 10–12 graph-derived features?

How should I split graphs into training and validation sets to test generalization across unseen architectures?

Help me visualize the difference between heuristic and GNN-predicted risk levels on the same graph and summarize the key differences.